

Unit 3- Process Management

Processes

What is a Process?

The program in execution.

A program is an inanimate entity; only when a processor “breathes life” into it does it become the “active” entity, we call a process.

Programs and Processes

A process is different than a program.

Consider the following analogy

Scenario-1: A computer scientist is baking a birthday cake for his daughter

Computer scientist - CPU

Birthday cake recipe - program

Ingredients - input data Activities: - processes

- reading the recipe
- fetching the ingredients
- baking the cake

Programs and Processes

Scenario-2: Scientist's son comes running in crying, saying he has been stung by a bee.

Scientist records where he - the state of running process was in the recipe saved

Reach first aid book and materials - Another process fetched

Follow the first aid action - Processor switched for new (high priority job) process

On completion of aid, cake - Completion of high priority baking starts again from job & return back to the last one where it was left

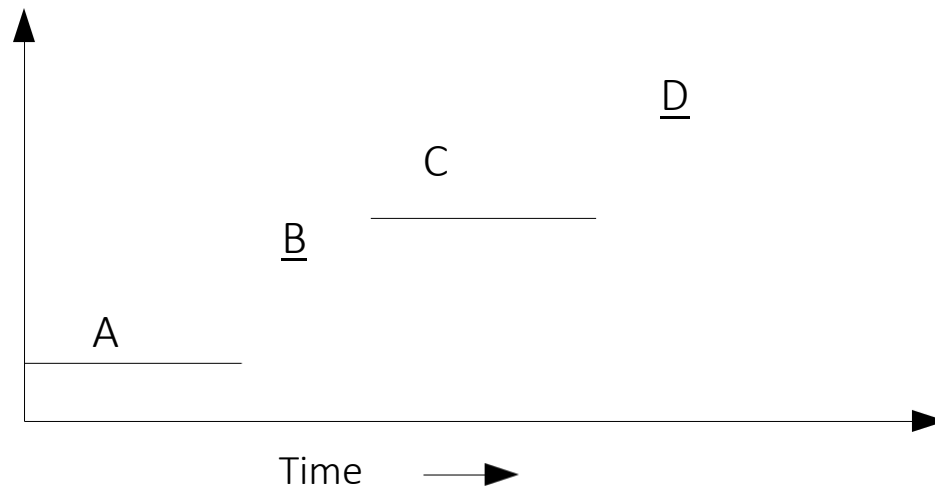
A process is an activity of some kind, it has program, input, output and state.

Process Models

- Uniprogramming
 - Multiprogramming
- Multiprocessing

Uniprogramming

Only one process at a time.



Examples: Older systems

Advantages: Easier for OS designer

Disadvantages: Not convenient for user and poor performance

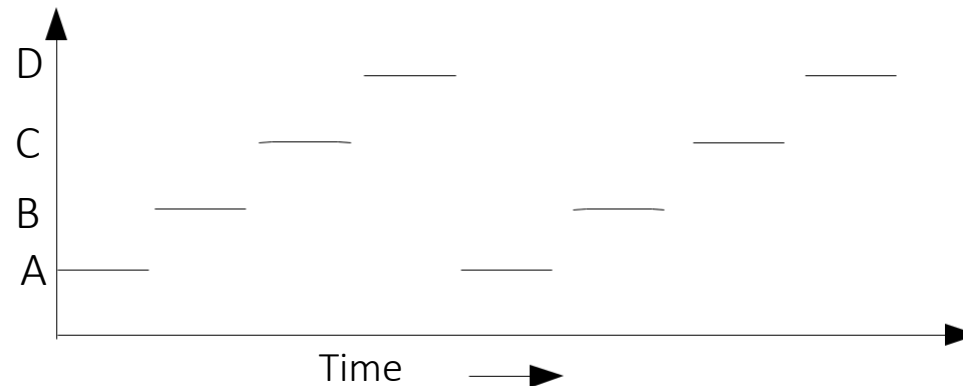
Multiprogramming

Multiple processes at a time.

OS requirements for multiprogramming:

Policy: to determine which process is to schedule.

Mechanism: to switch between the processes.



Examples: Unix , WindowsNT

Advantages: Better system performance and user convenience.

Disadvantages: Complexity in OS, Context Switching

Multiprocessing

*System with multiple processors.
Parallel Processing*



Process States

A process goes through a series of discrete process states.

Running state:

- Process executing on CPU. - Only one process at a time.

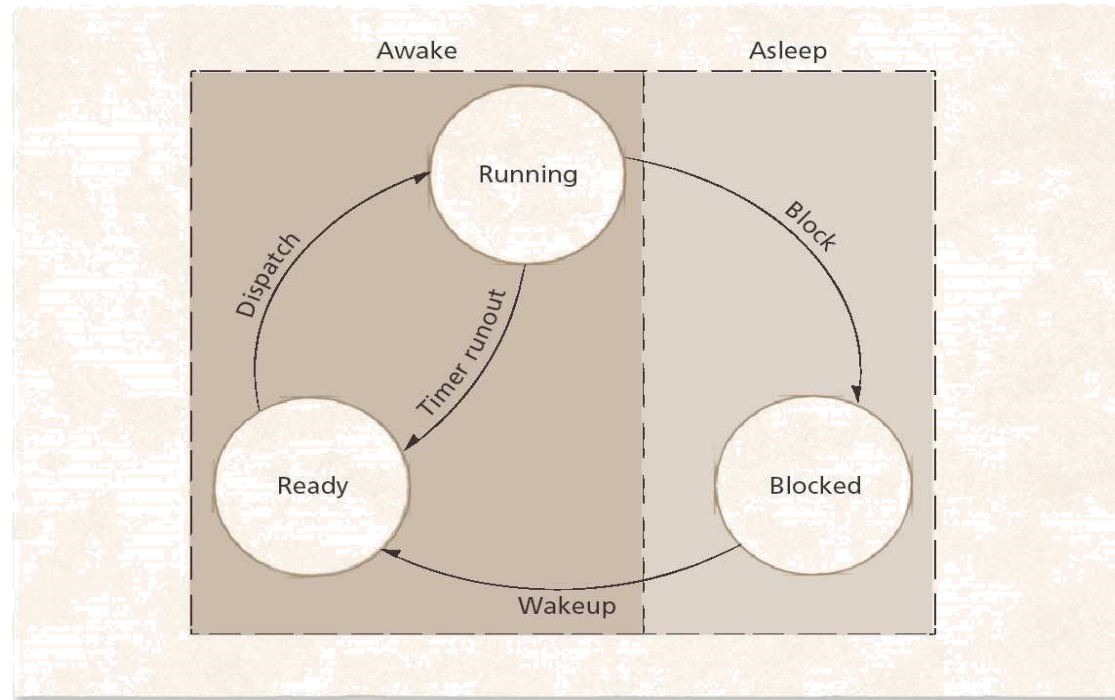
Ready state:

- Process that is not allowed to CPU but is ready to run.
- a list of processes ordered based on priority.

Blocked state:

- Process that is waiting for some event to happen (e. g. I/O completion events).
- a list of processes (no clear priority).

Process States



Process state transitions diagram

Process State Transitions

When a job is admitted to the system, a corresponding process is created and normally inserted at the back of the ready queue.

When the CPU becomes available, the process is said to make a state transition from ready to running.

`dispatch(processname): ready -> running.`

To prevent any one process from monopolizing the CPU, OS specify a time period (quantum time) for the process. When the quantum expire makes state transition running to ready.

`timerrunout(processname): running -> ready.`

When the process requires an I/O operation before quantum expire, the process voluntarily relinquishes the CPU and changed to the blocked state.

`block(processname): running -> block.`

When an I/O operation completes. The process make the transition from block state to ready state.

`wakeup(processname): blocked -> ready.`

Process Control Block

Process must be saved when the process is switched from one state to another so that it can be restarted later as it had never been stopped.

The PCB is the data structure containing certain important information about the process -also called process table or processor descriptor.

Process state: running, ready, blocked.

Program counter: Address of next instruction for the process.

Registers: Stack pointer, accumulator, PSW etc.

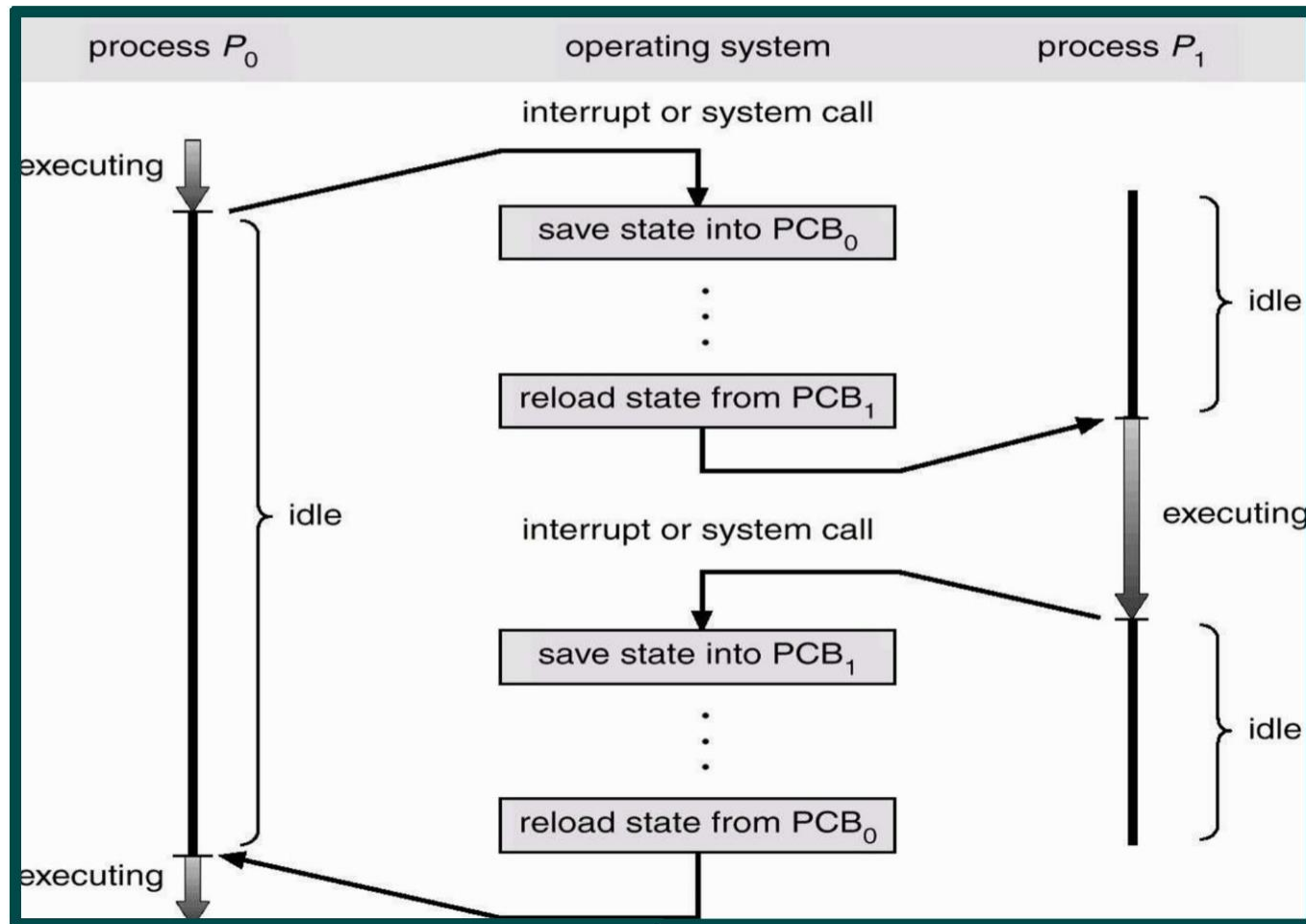
Scheduling information: Process priority, pointer to scheduling queue

Memory-allocation: value of base and limit register, page table, segment table

Accounting information: time limit, process numbers etc.

Status information: list of I/O devices, list of open files etc.

Process Control Block



Operations on Processes

The processes in the system can execute concurrently, and they must be created and deleted dynamically. OS provide the mechanism for process *creation* and *termination*.

Process Creation.

Process Termination.

Process Creation

There are four principal events that cause the process to be created:

- » System initialization.
- » Execution of a process creation system call.
- » User request to create a new process.
- » Initiation of a batch job.

A process may create several new processes during the course of execution. The creating process is called a parent process, where as the new processes are called children of that process.

Process Creation

Two ways to create a new process

1. Build a new one from scratch

- Load specified code and data into memory.
- Create and initialize PCB.
- Put processes on the ready list.

2. Clone an existing one (e.g. Unix `fork()` syscall)

- Stop the current process and save its state.
- Make copy of code, data, stack, and PCB.
- Add new process PCB to ready list.

Unix Process Creation Ex.

```
#include<stdio.h>
int main(int argc, char *argv[])
{
    int pid;
    pid = fork();          /* create new process */
    if(pid < 0)
    {
        /* error occurred */
        fprintf(stderr, "fork failed");
        exit(-1);
    }
    else if(pid == 0)
    {
        /*child process */
        execlp("/bin/ls", "ls",Null);
    }
    else
    {
        /* parent process */
        wait(Null);
        printf("Child Complete");
        exit(0);
    }
    return 0;
}
```


Process Termination

Process are terminated on the following conditions

1. Normal exit.
2. Error exit.
3. Fatal error.
4. Killed by another process.

Example:

In Unix the normal exit is done by calling a *exit* system call. The process return data (output) to its parent process via the *wait* system call. *kill* system call is used to kill other process.